

TRAINING PROGRAMME • 2026 EDITION

Product Engineering Mindset

Thinking like an owner, building like an engineer, working with AI as your pair

For freshers & final-year interns
Running case study: Personal Banking Management System

What you will take away today

Learning outcomes

- Explain the difference between building features and engineering products
- Write a use case, an epic, and a user story with testable acceptance criteria
- Pick outcome metrics (leading and lagging) for a real feature
- Describe a value stream and spot waste in it
- Scope an MVP and plan iterations
- Name the DevOps practices that let a team ship daily
- Use AI assistants safely to do all of the above faster

Agenda — 7 modules

- 1 What it means to be a product engineer
- 2 Quality-first thinking
- 3 Business outcomes
- 4 Value stream focus
- 5 User experience
- 6 Speed and agility
- 7 DevOps practices to scale

Running case: a Personal Banking app

One example used in every module, so the ideas connect



A bank wants customers to manage all their money from one app — balances, transfers, bills, budgets and support — without visiting a branch.



Customer pain today

- Branch visits for routine tasks
- No real-time view of spending
- Manual expense tracking
- Slow, confusing transfers
- Security worries online



Stakeholders

- Customers, small businesses, seniors
- Bank ops, support, compliance
- Payment partners, regulators



Capability areas

- Account management
- Transactions & payments
- Financial planning & insights
- Security & support



MODULE 1 OF 7

What it means to be a product engineer

You are not paid to write code. You are paid to make a user's problem go away — code is one of your tools.

Feature builder vs product engineer

Feature builder

- Waits for a ticket, builds exactly what it says
- Success = "my task is done"
- Never meets a user; guesses at intent
- Measures lines of code, story points
- Hands over to QA and Ops and moves on

Product engineer

- Asks "what problem, for whom, and why now?"
- Success = user behaviour changed, business moved
- Talks to users, reads support tickets, watches data
- Measures adoption, time-to-value, retention
- Owns build, quality, release and what happens after



Banking example: a ticket says "add a Transfer button". A product engineer asks: what does the customer do today, why is it slow, and how will we know the button helped?

The product engineer's loop

Every module today is one stop on this loop



Start with a use case, not a screen

- **Actor:** registered customer
- **Goal:** move money between own accounts securely
- **Main flow:** log in → pick accounts → enter amount → confirm → receipt
- **Alternate flows:** insufficient funds, invalid account, timeout



Questions a product engineer asks first

- Who exactly is the actor, and how often do they do this?
- What happens if we do nothing?
- What is the smallest version that proves value?
- How will we know it worked, in numbers?
- What must never break — money, security, trust?

Using AI to think like a product engineer

Where AI helps

- Draft stakeholder personas and their goals in minutes
- Turn a vague problem statement into 5–8 candidate use cases
- Generate alternate and error flows you would forget
- Summarise 200 support tickets into the top 5 pain points
- Act as a sceptical reviewer: "poke holes in this plan"

Where AI does not help

- It has never met your customer — it guesses plausibly
- It will confidently invent regulations and numbers
- It cannot decide what matters to the business
- It does not own the outcome. You do.



Prompt pattern: give context, ask for options, then challenge

"You are a product manager at a retail bank. Users: customers aged 25–45 who pay bills monthly. Problem: transfers take 9 steps. List 6 use cases ranked by frequency × pain, with one risk each." Then ask: "Which of these would a compliance officer reject, and why?"



MODULE 2 OF 7

Quality-first thinking

Quality is designed in and automated — not inspected in at the end by someone else.

Quality is built in, not inspected in

Shift left: the earlier you catch it, the cheaper it is

1×

fix cost while writing the story

10×

fix cost after code review

100×

fix cost in production

∞

cost of a customer losing trust



Definition of Done — team rule

- Acceptance criteria written before coding starts
- Unit + integration tests pass in CI, not just on my laptop
- Code reviewed by a human (AI review is extra, not instead)
- Security and privacy checklist ticked
- Monitoring / logs exist for the new path



Test pyramid — where effort goes

- **Many** fast unit tests (logic, validation, edge cases)
- **Some** integration tests (API + DB + payment gateway stub)
- **Few** end-to-end journeys (login → transfer → receipt)
- **Always** non-functional: security, performance, accessibility

Testable acceptance criteria — the banking story

Story 2.1 As a **security-conscious customer**, I want **alerts on suspicious transactions** so that I can **protect my account from fraud**.

✓ **Given** a transaction 200% above my normal spend **when** it is attempted **then** hold it and send SMS + email within 60 seconds

✓ **Given** I receive a fraud alert **when** I respond within 15 min **then** I can confirm or deny via a secure link

✓ **Given** I deny a transaction **when** the denial is recorded **then** the transaction is blocked and reported to the fraud team

Notice: every line has a number or a yes/no. If you cannot write a test for it, it is a wish, not a criterion.

Using AI for quality — with guard-rails

Generate tests

- Paste the function, ask for unit tests covering boundaries, nulls, currency rounding, concurrency
- Ask: "what input would break this?"

AI code review

- Run an AI reviewer in the pull request for style, obvious bugs, missing error handling
- Treat it as a first pass — a human still approves

Security scanning

- AI-assisted SAST / dependency scans explain findings in plain English
- Ask it to draft the fix and a regression test



Three rules for freshers using AI on real code

1) Never paste secrets, customer data or proprietary code into a tool your company has not approved. 2) AI-generated code is untrusted until you read it and tests pass — you sign the commit, not the model. 3) Ask for the reasoning ("why is this safe?"), not just the answer.



MODULE 3 OF 7

Business outcomes

Shipping is an output. A customer doing something differently, and the business benefiting, is an outcome.

Outputs vs outcomes



Outputs (what we shipped)

- Transfer feature released
- 12 stories closed this sprint
- 99.9% uptime
- Number of features per quarter



Outcomes (what changed)

- Transfers completed in 2 steps, not 9
- Branch visits for transfers down 60%
- Customer effort score up from 3.1 to 4.4
- Support calls about transfers halved

The mindset shift

From "How efficiently are we running our business?" to "How well are we solving our customers' problems — and does it show up in revenue, cost, or risk?"

Choosing metrics: leading vs lagging

Leading indicators let you act early; lagging ones tell you if it worked

Leading (early signals)

- Daily active users on the new flow
- Step-2 abandonment rate on transfers
- Feature discoverability within 7 days
- Support ticket volume about the feature

Lagging (results)

- Monthly retention / churn
- Net Promoter Score
- Revenue per customer
- Cost per transaction

HEART framework — a quick way to cover the whole customer experience

Happiness

satisfaction, NPS

Engagement

sessions / week

Adoption

% using new feature

Retention

% active after 90 days

Task success

completion rate,
errors

Banking MVP: what success looks like in numbers

≤ 2 min

Time to complete a transfer (was 9 steps)

$\geq 85\%$

Task completion rate for first transfer

24 h

Time-to-value: signup to first successful transaction



Customer Effort Score (CES)

- "How easy was it?" 1–5 after each key task
- Target ≥ 4.2 for transfers and bill pay



Digital resolution rate

- % of issues solved without a human agent
- Target 75% within 6 months



AI productivity tip

Ask an AI to build a metric tree: "Goal: fewer branch visits. Give me 3 leading and 3 lagging metrics, the formula for each, and the event I must log to compute it." Then paste your CSV export and ask it to write the pandas / SQL to compute them.



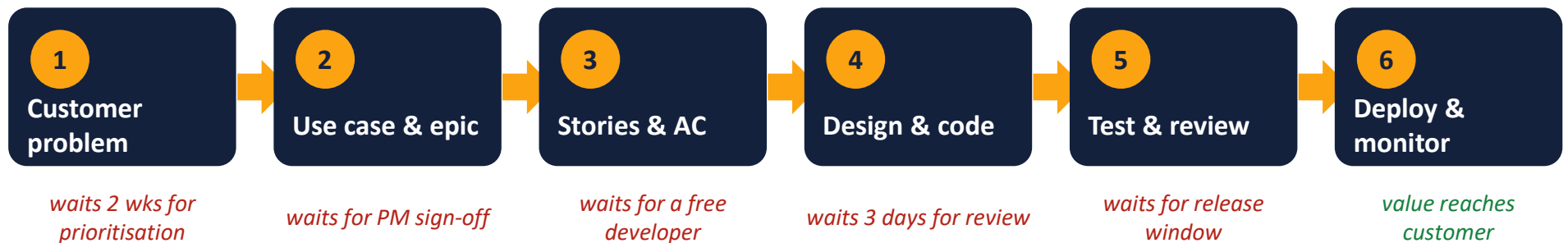
MODULE 4 OF 7

Value stream focus

Follow an idea from a customer's mouth to a running feature — and cut everything that does not add value on the way.

What a value stream looks like

Most of the elapsed time is waiting, not working



3 days

actual work time

6 weeks

elapsed lead time

7%

flow efficiency

🔍 Ask yourself

- Where does work sit idle?
- Which hand-off can we remove?
- What can be automated?

Typical numbers for a mid-size team — measure your own stream, do not assume.

Slicing value: epic → stories the team can flow

Epic 1 · Account Management As a customer I want to manage my accounts digitally so I can see my money anytime without visiting a branch.

1.1 View balances

Real-time balance across all accounts;
updates < 5 s

1.2 Transaction history

Last 90 days by default; filter by date,
amount, category

1.3 Download statements

PDF / CSV for any month; ready in <
10 s



INVEST — a good story is

Independent · Negotiable · Valuable · Estimable · Small · Testable

Small enough to finish in days, not weeks — that is what makes flow possible.



Vertical slices, not layers

- Bad: "build the database", "build the API", "build the UI"
- Good: "view balance for one account, end to end"

Using AI across the value stream

Discover	Cluster interviews and tickets into themes; draft problem statements
Define	Generate epics and INVEST stories as Markdown; convert to Jira / Linear / GitHub issues via integrations
Design	Produce sequence diagrams, API contracts and data models from the story text
Build	Agentic coding assistants scaffold code, migrations and tests from the spec — you review and steer
Verify	AI review on every pull request; explain failing tests; suggest missing edge cases
Operate	Summarise incidents, draft release notes and changelogs, answer "what changed in this deploy?"



Spec-driven development

Write the story + acceptance criteria first, hand that to the AI agent, and judge its output only against the criteria. A clear spec is now the highest-leverage engineering skill.



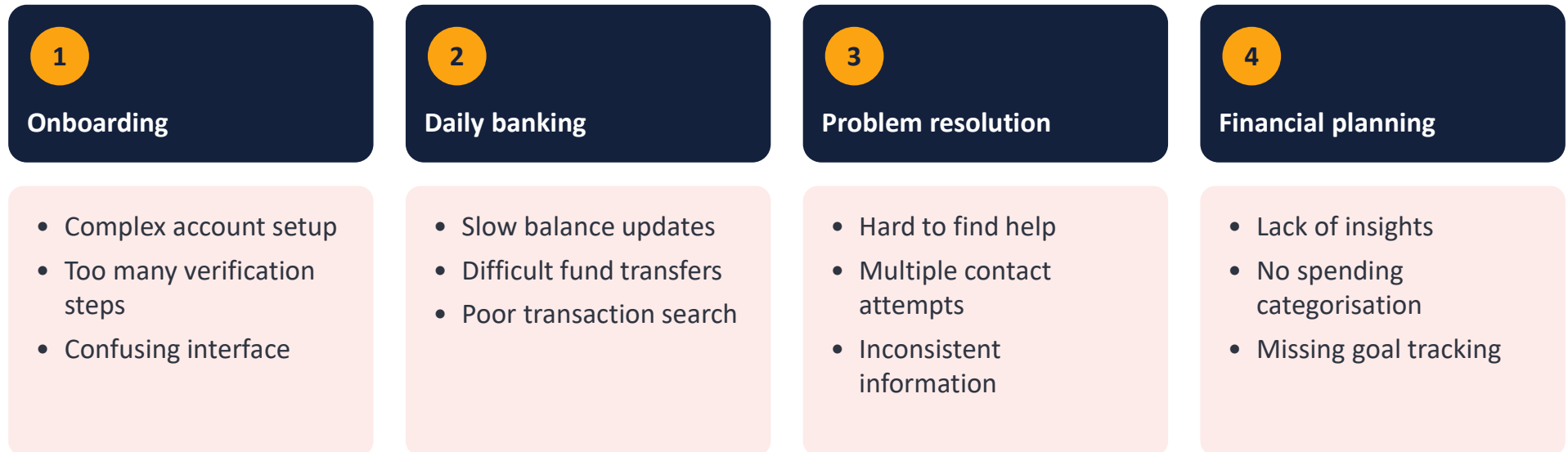
MODULE 5 OF 7

User experience

The customer does not see your architecture. They see whether they could pay their bill in 30 seconds.

Map the journey, find the pain

Personal banking customer journey



Group exercise (10 min)

For one stage, name 2 metrics (one leading, one lagging) and the single change that would most reduce the pain.

Scope the MVP: say no to almost everything

MVP = the smallest version that delivers core value and lets you learn

In the MVP (must-have)

- Secure login with MFA
- Account overview & balances
- Fund transfer (own + other bank)
- Transaction history (90 days)
- Bill payment (top 5 utilities)

MVP core

High impact · High frequency

Version 2

High impact · Low frequency

Nice to have

Low impact · High frequency

Avoid

Low impact · Low frequency

Not yet (why later)

- Investment platform — different user segment
- Loans — regulatory complexity
- Multi-currency — niche demand
- AI chat support — needs real usage data first

Common MVP mistakes

- Feature creep: "just one more feature before launch"
- Perfectionism: "it's not ready until it's perfect"
- Building for everyone instead of one segment

Using AI to design better experiences



Research synthesis

- Feed interview notes and app-store reviews; ask for themes with verbatim quotes
- Ask for a journey map draft you then verify with real users



Rapid prototypes

- Generate a clickable HTML prototype of the transfer flow in minutes
- Test 3 variants with 5 users before writing production code



Copy & accessibility

- Rewrite error messages in plain language, in Hindi and English
- Check colour contrast, screen-reader labels and reading level



AI inside the product — design it like any feature

An "AI chat support" feature still needs a use case, acceptance criteria (accuracy, escalation to a human, refusal on unsafe requests) and outcome metrics (digital resolution rate). Log every AI answer and rate it — that data is your next iteration.



MODULE 6 OF 7

Speed and agility

Speed is not typing faster. It is shortening the time between an idea and learning whether it was right.

Why agile beats waterfall for products

Waterfall

- Requirements → design → build → test → release, in sequence
- All learning arrives at the end, when change is most expensive
- Fits stable, well-understood problems (a bridge, a compliance report)

Agile / iterative

- Small increments, working software every 1–2 weeks
- Feedback from real users changes the next increment
- Fits products, where needs are discovered, not known

Build → Measure → Learn, applied to the banking MVP

Build

3-step transfer flow, 2 weeks

Measure

60% abandon at step 2, avg
3.2 min

Learn

Too many verification fields

Iterate

Reduce to 2 steps, add
progress bar

Plan iterations: decide with data, not opinions

Criteria	Weight	A · Budget tools	B · Investments	C · AI chatbot
Customer impact	40%	High (8)	Low (4)	Medium (6)
Development effort	30%	Low (3)	High (9)	Medium (6)
Market differentiation	20%	Medium (5)	High (7)	High (8)
Revenue impact	10%	Medium (5)	High (8)	Low (3)
Weighted score		6.4	5.7	5.9



Roadmap by outcome, not just by date

- v1 (m1–3): core banking — 10k users, 70% task completion
- v2 (m4–6): budgets & insights — 25k users, NPS > 50
- v3 (m7–9): planning tools — 50% use budgeting



Common pitfalls

- Ignoring the metrics you set and prioritising by gut feel
- Never talking to a user between releases
- "Sprint" that is just a mini waterfall

Speed in 2026: AI-assisted engineering

What has changed — and what has not

What is faster now

- Boilerplate, CRUD, migrations, test scaffolds: minutes, not days
- Reading an unfamiliar codebase: ask the agent to explain it
- Prototypes to put in front of users the same day
- Documentation and release notes generated from diffs

What is exactly as hard

- Knowing which problem is worth solving
- Deciding what to leave out
- Reviewing, testing and owning the result
- Security, privacy and regulatory judgement



Working with an agentic coding assistant (Claude Code, Copilot agent, Cursor and similar)

Give it the story + acceptance criteria + the test command. Ask for a plan before code. Review the diff like a senior would: naming, error paths, tests, secrets. Commit small. If you cannot explain a line it wrote, you are not done.



MODULE 7 OF 7

DevOps practices to scale

You build it, you run it. Automation and observability are what let a small team ship safely every day.

The delivery pipeline — from commit to customer



DORA metrics — how elite teams measure delivery

Deployment frequency
on demand, many per day

Lead time for changes
< 1 day commit →
production

Change failure rate
< 5% of deploys need a fix

Time to restore
< 1 hour

Banking example

A fix to the transfer validation bug reaches customers in hours, behind a feature flag for 5% of users first. If error rate rises, the flag is turned off in one click — no rollback deploy, no 2 a.m. call.

Practices that let a team scale



Infrastructure as code

Environments defined in Git (Terraform, Helm). Reviewable, repeatable, no snowflake servers.



Feature flags

Ship dark, release gradually, kill instantly. Decouples deploy from release.



Observability

Logs, metrics, traces for every new path. If you cannot see it, you do not own it.



Blameless post-mortems

Incidents produce learning and automation, not punishment.



Trunk-based & small PRs

Merge daily; a 200-line PR gets reviewed, a 2,000-line PR gets rubber-stamped.



Security as code

Secrets in a vault, dependency scanning in CI, least-privilege by default.



AI in operations (AIOps)

AI assistants now summarise an incident from logs and traces, propose the likely root cause, draft the runbook and the post-mortem. They are a great first responder — a human still decides to roll back.

Your AI productivity playbook — 8 habits

1

Context first

Role, users, constraints,
goal — then the ask

2

Ask for a plan

Before code, before a
document

3

Verify everything

Numbers, APIs, regulations
— AI invents confidently

4

Small loops

One story, one function,
one test at a time

5

Keep secrets out

No customer data or
credentials in unapproved
tools

6

Own the output

You sign the commit; the
model does not

7

Use it as a critic

"Find 3 flaws in this" beats
"is this good?"

8

Keep learning

Fundamentals let you
judge what AI gives you

Tools change every quarter; these habits do not.

Exercise: Insurance Policy Management System

Apply all seven modules to a new domain — 45 minutes, teams of 3

Epic 1 · Policy discovery & application

Compare products · Apply online · Instant quote

Epic 2 · Risk assessment

Integrated data · AI/ML risk model · IoT monitoring

Epic 3 · Claims processing & payment

File with documents · Review & approve · Settlement

Epic 4 · Insurance admin dashboard

Policy performance · Claims analytics · Regulatory reporting

Your team delivers

- Pick one epic; write 1 user story with 5 Given/When/Then criteria (INVEST)
- Define 1 leading + 1 lagging outcome metric with formulas
- Decide what is in and out of the MVP, with reasons
- List the 3 DevOps practices you need before launch

With AI — show your work

- Use an AI assistant for the draft
- Highlight 2 things it got wrong and how you caught them

Seven ideas to carry into your first project

- 1 Solve a problem for a person — code is the tool, not the goal
- 2 Quality is designed in early and automated; it is everyone's job
- 3 Measure outcomes (behaviour, money, risk) not outputs (features shipped)
- 4 Watch where work waits; shorten the stream from idea to customer
- 5 The user's experience is the product; scope ruthlessly
- 6 Speed = shorter learning loops, not longer hours
- 7 Automate delivery and observe production — you build it, you run it

And in every one of them: use AI to go faster, keep the judgement and the ownership yourself.